

Assignment 2

Factory Method and Abstract Factory

Astana IT University | School of Software Engineering

Course: ShP-2216 Software Design Patterns | Academic year 2026-2027

Programme: 6B06102 Software Engineering | Year 2, Trimester 4

Instructor: Yerassyl Bekenov

Format: Individual work | Java, JDK 17 | Report in English

Weight: 15% of the 1st Attestation grade

Submission Deadline: 20.09.2026, 23:59

Defense: Until your group's last practice class of Week 3.

1 Your task

Build one Java logistics application that supports road and sea delivery and can use Windows or macOS UI components. Apply Factory Method to create the transport and Abstract Factory to create a matching button and checkbox. Both parts must run together in the same program.

A console application is sufficient: UI components may print their platform and component type. A graphical window, database, web framework, and external services are not required. You may use IntelliJ IDEA or another Java IDE.

2 Learning objectives

- Explain the roles and differences of Simple Factory, Factory Method, and Abstract Factory (CLO 1).
- Implement both required patterns and represent their structure in UML (CLO 2).
- Demonstrate the application and justify your design decisions (CLO 3).

3 Part A Factory Method

Implement the following roles. Use the names below so that your implementation can be checked consistently.

Pattern role	Required implementation
Product	Transport: an interface or abstract class defining delivery behavior.
Concrete products	Truck and Ship: visibly different road and sea delivery behavior.
Creator	Abstract Logistics: declares <code>createTransport()</code> returning Transport, and provides a shared <code>planDelivery(...)</code> method.
Concrete creators	RoadLogistics and SeaLogistics: each overrides <code>createTransport()</code> and creates its corresponding transport.

The shared `planDelivery(...)` method must obtain a transport through `createTransport()` and call its delivery behavior through the Transport contract. Pass a cargo description and destination into the delivery workflow; fixed sample values are acceptable.

A single `create(type)` method containing only an `if` or `switch` is a Simple Factory. It does not satisfy Part A: your solution must include the creator subclasses and the overridden factory method.

4 Part B Abstract Factory

Create a UI component family for each platform. Each family must contain both product types, and both types must be used by the application.

Product type	Windows family	macOS family
Button	WindowsButton	MacOSButton
Checkbox	WindowsCheckbox	MacOSCheckbox

- Define separate Button and Checkbox interfaces, each with a rendering method such as `paint()`.
- Define a GUIFactory interface with `createButton()` and `createCheckbox()`. Each method must return the corresponding product interface.
- Implement WindowsFactory and MacOSFactory. Each factory must create a complete, matching pair of components for its own platform.
- Implement a DeliveryApplication client that receives a GUIFactory and a Logistics object through its constructor. It must obtain both UI components from that factory and use them through their interfaces.

For example, choosing WindowsFactory must produce a Windows button and a Windows checkbox in the same run. The client must not assemble a pair by directly calling concrete component constructors.

5 Combine the two patterns

At runtime, accept two independent choices using console input or command-line arguments: delivery mode (ROAD or SEA) and UI platform (WINDOWS or MACOS). All four combinations must work without editing the code.

- In Main or one startup helper, validate the choices, select the concrete logistics creator and GUI factory, and pass them to DeliveryApplication.
- In the application, render the button and checkbox, then call the logistics delivery workflow. Print enough information to identify the chosen transport, cargo, destination, and UI family.
- For an unsupported or missing choice, display a clear message and either ask again or stop cleanly. Do not silently use a default or continue with a null factory.

A small `if` or `switch` at startup is allowed to select creators and factories. Delivery and rendering logic must use the abstract contracts without branching on concrete product classes or using casts or `instanceof` to select their behavior.

Example of observable output

The wording is illustrative; your output may differ if it clearly demonstrates the same behavior.

```
Delivery mode: ROAD
UI platform: WINDOWS
Rendering Windows button
Rendering Windows checkbox
Truck delivers laboratory equipment to Aktau warehouse
```

Changing only the platform must replace both UI components while preserving the selected delivery mode. Changing only the delivery mode must replace the transport while preserving the UI family.

6 Required verification

Run all six checks below and record the input, actual result, and pass or fail status in your report. Console transcripts or readable screenshots are acceptable evidence. Automated tests are optional.

Check	Input	Expected result
1	ROAD + WINDOWS	Truck delivery; Windows button and checkbox.
2	SEA + WINDOWS	Ship delivery; Windows button and checkbox.
3	ROAD + MACOS	Truck delivery; macOS button and checkbox.
4	SEA + MACOS	Ship delivery; macOS button and checkbox.
5	Unsupported delivery mode with a valid platform	Clear validation message; no delivery with an invalid selection.
6	Unsupported platform with a valid delivery mode	Clear validation message; no UI construction with an invalid selection.

Also confirm that missing input follows your documented validation behavior. All supported configurations must run on one machine; selecting macOS components must not require a computer running macOS.

7 Clean Code requirements

Apply and explain at least five Clean Code practices. For each one, include a short annotated excerpt from your own implementation and explain the specific benefit. A before-and-after excerpt may be used when you refactor a problem.

- Meaningful names: names distinguish products, creators, factories, and the application client.
- Small methods: input handling, object creation, rendering, and delivery each have a clear responsibility.
- Avoid duplicated logic: share the delivery workflow and keep repeated object creation in the appropriate creator or factory.
- Data abstraction from Chapter 6: callers use the transport and UI contracts without depending on concrete implementations or exposed internal state.
- Objects and encapsulation from Chapter 6: use methods such as delivery and rendering to express behavior; keep internal state private where it is needed and avoid unnecessary accessors.

Include at least two explanations connected to Clean Code, Chapter 6. The last two items above satisfy that focus when supported by your code. Organize the two pattern implementations into clear Java packages and keep the project easy to compile and run.

Design reflection

In the report, explain what would change when adding one more transport, one more UI family, and one more UI product type such as a text field. Identify which interfaces or classes would need changes and which existing client logic could remain unchanged. You only need to explain these extensions; implementing them is not required.

8 What to submit

GitHub repository

Submit one repository containing both parts of the application. Include:

- Complete Java source code and any build files required to run it with JDK 17.
- A README.md with the project purpose, package structure, prerequisites, exact build and run instructions, supported input values, and one sample run.
- UML diagram files and an incremental commit history showing meaningful stages of your work.

Check that the instructor can access the repository. A private repository is acceptable if access has been granted to the instructor before the submission deadline.

Report in Moodle

Upload one PDF report in English named Assignment2_Group_Surname_Name.pdf, replacing the name and group fields with your details. A concise report of about 3-5 pages is recommended; add pages if needed to keep diagrams and evidence readable.

- Identification: full assignment title, your full name, group, and GitHub repository link.
- Introduction: what your application does and why each pattern fits its part of the task.
- Two UML class diagrams: one for Factory Method and one for Abstract Factory. Label the pattern roles, important methods, inheritance or implementation relationships, and client dependencies. The diagrams must match the submitted code.
- Clean Code evidence: five annotated excerpts and explanations as required in Section 7.
- Verification evidence: the six checks in Section 6 with actual results, plus the missing-input behavior.
- Conclusion: compare the patterns and provide the design reflection from Section 7. List the references you used.

Submit the repository link in Moodle together with the PDF before the Week 2 deadline. Identify the submitted commit in the report or README so that the version being assessed is clear. Before uploading, reopen the PDF and run the project using your README instructions.

9Assessment

Total: 100 points. Report: 10; working and clean code: 40; individual defense: 50. The assignment contributes 15% to the 1st Attestation grade.

Award the points attached to each completed check below. A check earns its stated points only when all its requirements are met; otherwise it earns zero. Full credit means all checks in that row are met. Partial credit is the sum of completed checks, and zero means none are met. Do not apply additional deductions outside this rubric and the course policies in Section 11, except for the code-comprehension deduction defined under Individual defense below, which the instructor may apply during the defense in addition to the D1-D6 checks.

Report 10 points

Criterion	Max	How points are earned
R1 UML diagrams	4	1 point for each check: FM diagram: all product and creator roles with correct inheritance or implementation relationships. FM diagram: factory method, shared workflow, and dependencies match the code. AF diagram: both product types, both families, factory roles, and correct relationships. AF diagram: creation methods and client dependencies match the code.
R2 Explanations	4	1 point for each check: Correct rationale for using both patterns in this application. All three extensions in Section 7 explained with the affected classes or interfaces identified. Five valid excerpts from the submitted code, each with a named Clean Code practice and a specific benefit. At least two of those explanations correctly connect the code to Chapter 6.
R3 Evidence and submission	2	1 point: input, actual result, and pass or fail status recorded for all six required checks, plus missing-input behavior. Honestly recorded failures still earn this report point. 1 point: PDF naming and identification, accessible repository link, submitted commit, README, incremental commit history, and references are provided as required in Section 8.

Working and clean code 40 points

Criterion	Max	How points are earned
C1 Factory Method	15	Contracts and products (3): 1 for a Transport delivery contract; 1 each for Truck and Ship implementing the contract with the required delivery behavior. Creators (6): 1 for abstract Logistics; 1 for its createTransport() declaration returning Transport; 2 per concrete creator, split into 1 for a correct override and 1 for creating the correct transport. Shared workflow (4): 1 for planDelivery(...) in Logistics; 1 for obtaining the product through the factory method; 1 for invoking delivery through Transport; 1 for passing and using cargo and destination. Client (2): 1 for a Logistics-typed dependency; 1 for calling its workflow without constructing or branching on concrete transport types. A Simple Factory without creator subclasses overriding the factory method can earn only the 3 product points in C1.
C2 Abstract Factory	15	Product interfaces (2): 1 each for Button and Checkbox with a rendering contract. Concrete products (4): 1 each for the four required classes implementing the correct interface and rendering their own platform and component type. GUIFactory (3): 1 for the factory interface; 1 each for correctly typed createButton() and createCheckbox() declarations. Concrete factories (4): 1 per creation method across both factories, provided the factory implements GUIFactory and returns the correct product from its own family. Client (2): 1 for constructor injection of GUIFactory and obtaining both components from it; 1 for rendering both through interfaces without concrete constructors or casts.
C3 Integration and validation	5	1 point: the submitted project builds and runs on JDK 17 using the README instructions. 1 point: 0.25 for each of the four valid runtime combinations producing the correct delivery and matching UI pair in one application, without editing code. 3 points: 1 each for handling an unsupported delivery mode, an unsupported platform, and missing input with a clear message and a clean stop or retry. No silent default, crash, or continuation with an invalid choice.
C4 Clean Code in the implementation	5	1 point for each practice demonstrated in the code: - Names consistently identify the domain and pattern roles. - Setup, product creation, rendering, and delivery have separate responsibilities. - The shared delivery workflow and object creation logic are not duplicated. - Client dependencies use abstractions without exposing concrete product internals. - Internal state, if present, is private; accessors have a clear use. Stateless classes are acceptable.

If the submitted application cannot be run, C3 earns zero. Assess the report and other code checks using the evidence actually available; a build failure does not automatically make the whole assignment zero. Missing or incorrect evidence earns no points for the affected check.

Individual defense 50 points

Use the submitted version during the defense. For D1 and D2, each role check is worth 2 points: 1 for an accurate explanation and 1 for locating and explaining the matching code. A memorized definition alone can earn only the explanation point. The live or recorded demonstration in Section 9 is accepted for D4.

Criterion	Max	How points are earned
D1 Factory Method roles	10	2 points for each of five checks: Product; Concrete Products; Creator; Concrete Creators; and the runtime path from planDelivery(...) through createTransport() to delivery. Identify the actual classes and calls in your project.
D2 Abstract Factory roles	10	2 points for each of five checks: Abstract Products; Concrete Products grouped into families; Abstract Factory; Concrete Factories; and the client using the injected factory and product interfaces.
D3 Comparison and extension decisions	15	Simple Factory versus Factory Method (5): 1 each for explaining conditional creation, subclass overrides, the shared product contract, your implementation, and why startup selection is allowed. Factory Method versus Abstract Factory (5): 1 each for explaining a product hierarchy, related product families, the transport use case, the GUI use case, and family consistency. Extensions (5): 1 each for identifying changes for a new transport, a new UI family, and a new UI product type; startup selection updates; and which client logic changes or stays unchanged.
D4 Demonstration	5	1 point for showing each outcome: road delivery; sea delivery; a complete Windows UI pair; a complete macOS UI pair; and one correctly handled invalid-input case.
D5 Explanation of own code	5	For one method selected by the instructor, earn 1 point each for correctly explaining its purpose, inputs and return value (including none or void), control flow, collaborating objects or interfaces, and the effect of one hypothetical input or code change.
D6 Clean Code justification	5	1 point per accurate explanation of a Section 7 practice, supported by your own code and its benefit. At most three points are for general practices; the other two require Chapter 6 examples.

Code comprehension deduction. If, at any point during the defense, a student cannot explain how a part of their own submitted code works, or shows only a partial or superficial understanding of it, the instructor may deduct up to 20 points from the total Individual defense score, in addition to the D1-D6 checks above. This applies to any part of the submission the instructor chooses to probe, not only the method selected for D5, and regardless of whether that code runs correctly, compiles, or was praised elsewhere in the rubric. The deduction reflects the requirement in Section 9 that the student can explain, in their own words, the code they submitted as their own work; it is not a substitute for the D1-D6 checks and does not replace them. The instructor determines the size of the deduction based on how much of the code is not understood and how central that part is to the required patterns (Sections 3-5).

Partial credit examples: D1 earns 5/10 for three accurate explanations and two matching code examples; C3 earns 0.75/1 for three valid configurations.

Add report, code, and defense points, keeping fractions and applying Section 11. A score of 80/100 adds 12 attestation points. Achievement bands: Excellent 90-100; Good 70-89; Satisfactory 50-69; Fail 0-49.

10 Submission and academic integrity policies

Defense is mandatory. Under the syllabus, failure to defend results in zero for the entire assignment, even if the code and report were submitted.

Late submission. Work submitted after the deadline receives zero unless an extension has been approved for an exceptional circumstance under the course and university rules. Contact the instructor as early as possible if such a circumstance arises.

Individual work. You may discuss concepts with classmates, but write your code and report independently. Cite borrowed ideas, explanations, and any referenced code. Copying another student's solution or submitting work that is not your own is handled under the university academic integrity policy.

Generative AI. The syllabus permits AI tools only as a reference for examples and clarification. They must not replace your independent work. You are responsible for understanding and explaining everything you submit.

12 Learning resources

- Lecture 2: Factory Method and Abstract Factory, including the Logistics and GUI examples and the Clean Code, Chapter 6 slides.
- Freeman and Robson, Head First Design Patterns, Chapter 4.
- Robert C. Martin, Clean Code, Chapter 6, Objects and Data Structures.
- Software Design Patterns syllabus 2026-2027 and the official course calendar in Moodle for submission dates and course policies.